

ECE/CS/ME 539: Activity

Multi-Layer Perceptrons

Solution

Problem 1: Binary Operations as Single-Layer Perceptrons

Let $x_1, \dots, x_M$ be a set of M binary-valued inputs, $x_m \in \{0, 1\}$, to a perceptron model that has a threshold (aka step) activation function. Find a $M + 1$ weight vector $\mathbf{w}^\top = [w_0 \ w_1 \ \dots \ w_M]$ to implement each of the following Boolean functions. Do not forget the bias input is a constant 1 and all $M + 1$ entries of $\mathbf{w}$ need to be specified in each case. Answers are NOT unique.

- (a) $f(x) = x_1 \text{ AND } x_2 \text{ AND } x_3 \dots \text{AND } x_M$.

Hint: $y = 1$ only if all $x_m = 1$. Otherwise, $y = 0$.

Answer: $\mathbf{w} = [-M + 0.5, 1, 1, \dots, 1]^T$. Since

$$g(\mathbf{x}) = (-M + 0.5) + \sum_{m=1}^M 1 \cdot x_m \text{ where } x_m \in \{0, 1\}$$

Only if ALL $x_m = 1 \Rightarrow g(\mathbf{x}) = 0.5$ and $y = 1$. Otherwise, if any $x_m = 0 \Rightarrow g(\mathbf{x}) < -0.5$ and hence $y = 0$.

- (b) $y = x_1 \text{ OR } x_2 \text{ OR } x_3 \text{ OR } \dots \text{OR } x_M$.

Hint: $y = 0$ only if all $x_m = 0$. Otherwise, $y = 1$.

Answer: $\mathbf{w} = [-0.5, 1, 1, \dots, 1]^T$. Since

$$g(\mathbf{x}) = -0.5 + \sum_{m=1}^M 1 \cdot x_m \text{ where } x_m \in \{0, 1\}$$

Only if ALL $x_m = 0$ one has $g(\mathbf{x}) = -0.5$ and $y = 0$. Otherwise, $y = 1$.

- (c) $y = \text{NOT } x_1$. (Assume $M = 1$)

Answer: $\mathbf{w} = [0.5, -1, 0, \dots, 0]^T$. Since $w_2 = \dots = w_M = 0$, inputs $x_2, \dots, x_M$ will have no impact on $g(\mathbf{x})$ nor y . $g(\mathbf{x}) = 0.5 - x_1$ where $x_1 \in \{0, 1\}$. Therefore, when $x_1 = 0$, $g(\mathbf{x}) = 0.5 \Rightarrow y = 1$, and when $x_1 = 1$, $g(\mathbf{x}) = -0.5 \Rightarrow y = 0$.

- (d) $y = \begin{cases} 1 & \text{if } \sum_m x_m \geq \frac{M}{2} \\ 0 & \text{otherwise} \end{cases}$

This is the majority vote function.

Answer: $\mathbf{w} = [-M/2, 1, 1, \dots, 1]$

Problem 2: Binary Addition as a Multi-Layer Perceptron

Implement a two-layer neural network to perform binary addition. The network should take two 2-bit binary sequences as inputs, a_1a_0 and b_1b_0 , and output a 3-bit binary sequence, $c_2c_1c_0$, where the output represents the sum of the two input sequences.

- The first layer should use a ReLU activation function.
- The second (output) layer should use a step function (binary threshold) as its activation function.

Specifications

- Inputs: Two 2-bit binary sequences, a_1a_0 and b_1b_0 , where $a_1, a_0, b_1, b_0 \in \{0, 1\}$.
- Output: A 3-bit binary sequence, $c_2c_1c_0$, where the output represents the binary sum of the input sequences, i.e.,

$$c_2c_1c_0 = a_1a_0 + b_1b_0.$$

- Layer 1: Implements a ReLU activation function on the inputs.
- Layer 2: Implements a step function (binary threshold) to produce the final output.

Answer (NOTE: The solution is not unique.)

Solution

$$\begin{aligned} c_0 &= \text{Step}(\text{ReLU}(a_0 + b_0 - 0.5) - 2 * \text{ReLU}(a_0 + b_0 - 1)) \\ c_1 &= \text{Step}(\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 1.5) - 2\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 2.5) \\ &\quad + 4\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 5.0) - 6.5\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 6.5)) \\ c_2 &= \text{Step}(\text{ReLU}(2a_1 + a_0 + 2b_1 + b_0 - 3.5)) \end{aligned}$$

where Step is step function: $\text{Step}(x) = 0$ if $x \leq 0$; otherwise 1. $\text{ReLU}(x) = 0$ if $x \leq 0$; otherwise x

Explanation

Since $c_0 = 1$ when $a_0 \oplus b_0 = 1$, we can express c_0 as:

$$\text{Step}(\text{ReLU}(a_0 + b_0 - 0.5) - 2 \times \text{ReLU}(a_0 + b_0 - 1))$$

The decimal number represented by $c_2c_1c_0$ can be computed as $2(a_1 + b_1) + a_0 + b_0$. The value $c_1 = 1$ when the decimal number is 2, 3, or 6. Therefore, we can use the expression, which is shown in the figure 1

$$\begin{aligned} c_1 &= \text{Step}(\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 1.5) - 2\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 2.5) \\ &\quad + 4\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 5.0) - 6.5\text{ReLU}(2(a_1 + b_1) + a_0 + b_0 - 6.5)) \end{aligned}$$

For c_2 , which equals 1 when the decimal number is greater than 3, we use:

$$\text{Step}(\text{ReLU}(2a_1 + a_0 + 2b_1 + b_0 - 3.5))$$

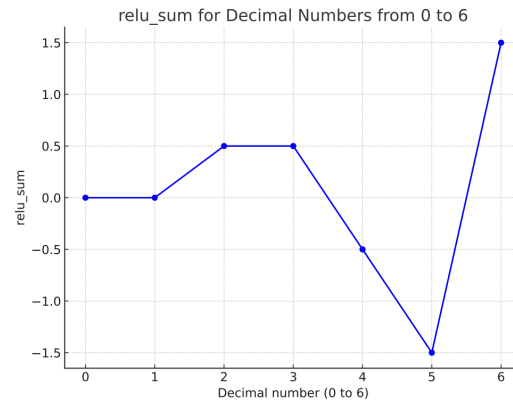
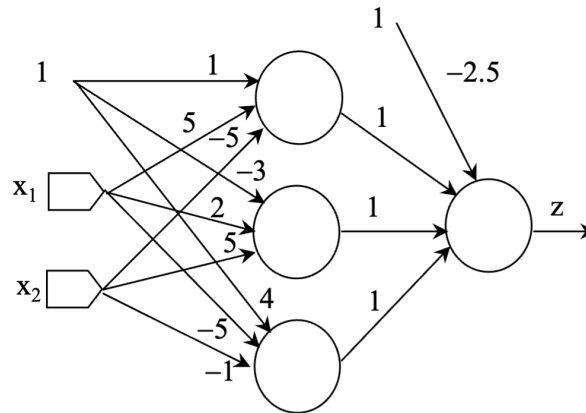


Figure 1: ReLU Sum for Decimal Numbers from 0 to 6

Problem 3: Multi-Layer Perceptron in PyTorch

Consider the 2-layer perceptron shown in the figure where the weights and bias are labeled. Assume all neurons use the sigmoid activation function. With the help of the provided starter code, implement this feed forward neural network.



- Start by specifying the weight matrices and biases.
- Implement both the hidden and output layer, in order to compute the model output for each datapoint specified in matrix X .
- Visualize all three hidden neurons (after the sigmoid activation) as well as the outputs using a 3d surface plot. Note that the starter code defines the input X to be a grid in the 2D plane. The surface will show the output of each neuron at different points in the 2D plane. How do the directions of increase of the hidden neurons relate to the weight matrix?
- After inspecting the visualization for the output neuron, we observe that the output for the datapoint $x=(1, -1)$ is low (compared to other datapoints). Propose a change of the model weights that would increase the model output at that point. Verify your proposal by running the model with the different weights.

Answer: See Ex_FFNet_sol.ipynb